

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте

на тему: _____ чат-бот для абитуриентов _____

Выполнил:

Студент группы БПМИ2412 _____

Веляев

Подпись

А.В.Веляев

И.О.Фамилия

01.06.2026

Дата

Принял:

Руководитель проекта

Андреева Дарья Александровна

Имя, Отчество, Фамилия

Старший инженер нейронных сетей

Должность, ученое звание

ООО "ИТ ИКС 5 ТЕХНОЛОГИИ"

Место работы (Компания или подразделение НИУ ВШЭ)

Москва 2026

Содержание

1	Введение	4
1.1	Актуальность	4
1.2	Цели и задачи	4
2	Обзор литературы	4
2.1	Подходы к построению вопросно-ответных систем	5
2.2	Обоснование выбора RAG	5
3	Архитектура системы	6
4	Ход работы	7
4.1	Описание датасетов	7
4.1.1	База знаний	7
4.1.2	Тестовый датасет	7
4.2	Выбор метрик	8
4.3	Подготовка данных и метод чанкирования	8
4.3.1	Предобработка текстов	8
4.3.2	Метод чанкирования	9
4.4	Выбор модели векторного представления текста	9
4.5	Выбор модели переранжирования	9
4.6	Сравнительный анализ конфигураций пайплайна	10
4.6.1	Конфигурация 1: Base RAG	10
4.6.2	Конфигурация 2: Добавление Reranker	10
4.6.3	Конфигурация 3: Добавление Rewrite	10
4.6.4	Конфигурация 4: Добавление LLM Decomposition	11
4.6.5	Конфигурация 5: Добавление FAQ-кеша	11
4.6.6	Конфигурация 6: Добавление Agentic Loop	11
4.6.7	Итоговый пайплайн	11
4.7	Архитектура Telegram-бота	11
4.8	Результаты и анализ	12
5	Заключение	13
6	Использование ИИ	13
	Список литературы	14

Аннотация

В данной работе представлена разработка и оценка Telegram-бота для автоматизации консультирования абитуриентов факультета компьютерных наук НИУ ВШЭ. Система построена на основе расширенного RAG-пайплайна (Retrieval-Augmented Generation) с многоуровневой обработкой запросов: детерминированное переформулирование, LLM-декомпозиция на подзапросы, векторный поиск в ChromaDB с эмбедингами multilingual-E5, переранжирование cross-encoder'ом и генерация ответа моделью YandexGPT. Для снижения нагрузки на языковую модель реализован трёхуровневый FAQ-кеш с точным, нечётким и динамическим сопоставлением, который автоматически пополняется ответами, подтверждёнными пользователями, и очищается от некачественных записей. Качество системы оценено на тестовом наборе вопросов по метрикам Hit@5, MRR@5, Precision@5, Context Recall и ROUGE-L. Результаты подтверждают эффективность предложенного подхода для задач предметно-ориентированного консультирования.

Ключевые слова: RAG, векторный поиск, ChromaDB, multilingual-E5, cross-encoder, переранжирование, декомпозиция запросов, YandexGPT, Telegram-бот, НИУ ВШЭ.

1 Введение

Самостоятельный анализ документов занимает у поступающих много времени, а нагрузка на сотрудников учебного офиса в пик приемной кампании резко растет. Был проанализирован официальный Telegram-чат «ФКН ПМИ 2025» для абитуриентов, и оказалось, что только в нем пользователи задали около 700 вопросов. Можно было бы спросить большую языковую модель, однако в таком случае есть риск получить недостоверную информацию из-за склонности LLM к галлюцинациям. Чтобы модель выдавала точные ответы, ей необходим контекст. Именно поэтому в рамках проектной работы я реализовал Telegram-бота на основе архитектуры RAG, который сможет отвечать на вопросы абитуриентов круглосуточно и исключительно по официальным документам. В работе описаны мотивация выбора архитектурного подхода, принципы формирования базы знаний из официальных документов ФКН, а также методы повышения качества поиска и генерации ответов. Отдельное внимание уделено проектированию пользовательского интерфейса и механизмам фильтрации нецелевых запросов.

1.1 Актуальность

В периоды приёмных кампаний учебный офис факультета компьютерных наук НИУ ВШЭ сталкивается с резким ростом однотипных обращений абитуриентов, ответ на каждое из которых требует обращения к нормативной документации. Это приводит к увеличению времени ожидания и высокой нагрузке на сотрудников. Автоматизация обработки типовых запросов посредством чат-бота позволяет снизить эту нагрузку и обеспечить абитуриентам мгновенный доступ к актуальной информации в любое время суток – в том числе в нерабочие часы и выходные дни.

1.2 Цели и задачи

Цель работы – разработка и оценка интеллектуальной системы консультирования абитуриентов ФКН НИУ ВШЭ на основе архитектуры RAG, обеспечивающей достоверные ответы исключительно из официальных документов в режиме 24/7.

Задачи:

1. Проанализировать существующие подходы к построению RAG-систем и обосновать выбор архитектурных решений для предметно-ориентированного консультирования.
2. Собрать и структурировать базу знаний из официальных документов факультета, обеспечив её полноту и актуальность для нужд приёмной кампании.
3. Исследовать методы улучшения качества поиска – переформулирование и декомпозицию запросов, переранжирование – и реализовать их в рамках единого пайплайна.
4. Разработать механизм кеширования, снижающий зависимость системы от языковой модели при обработке повторяющихся запросов.
5. Реализовать пользовательский интерфейс, обеспечивающий безопасное и удобное взаимодействие с системой в мессенджере Telegram.
6. Провести количественную оценку качества системы и сравнить эффективность отдельных компонентов пайплайна.

2 Обзор литературы

Разработка систем автоматического ответа на вопросы пользователей представляет собой активно развивающуюся область исследований, в которой сложились несколько принципиально различных подходов. В данном разделе рассматриваются основные из них применительно к задаче настоящего проекта – созданию чат-бота для ответов на вопросы абитуриентов НИУ ВШЭ на основе официальных документов.

2.1 Подходы к построению вопросно-ответных систем

Исторически первым решением стали кнопочные боты с фиксированными сценариями. Они детерминированы, и их легко тестировать, но они не способны обрабатывать произвольные вопросы пользователей. Кроме того, при любом изменении данных приходится вручную переписывать всю логику ветвления. Современные исследования в области обработки естественного языка (NLP) предлагают два альтернативных подхода к интеграции внешних знаний. Первый из них — дообучение (Fine-tuning), при котором базовую модель дополнительно тренируют под конкретную задачу на собственных датасетах. Однако на практике этот путь сталкивается с серьезными техническими трудностями. Современные LLM обучаются на огромных датасетах с использованием сложной инфраструктуры, поэтому корректно встроить новые факты в веса модели крайне непросто. Помимо высоких требований к вычислительным мощностям, у Fine-tuning есть ключевой недостаток: знания модели остаются статичными, а значит, любое обновление регламентов заставит запускать полный цикл переобучения с нуля. В условиях регулярно актуализируемых правил приёма это делает дообучение практически неприменимым. Второй подход — Retrieval-Augmented Generation (RAG) — устроен иначе. Вместо заучивания фактов на этапе предобучения модель извлекает нужные данные из внешнего хранилища в момент запроса. Процесс выглядит следующим образом: пользовательский запрос переводится в векторное представление (эмбеddинг), после чего система находит в базе релевантные фрагменты документов по семантическому сходству. Этот найденный контекст передается языковой модели, которая и формулирует итоговый ответ. Главное преимущество RAG заключается в гибкости работы с данными: чтобы актуализировать информацию, не нужно трогать саму модель — достаточно просто заменить или дополнить документы в текстовом хранилище.

2.2 Обоснование выбора RAG

С учётом требований и ограничений настоящего проекта архитектура RAG представляется наиболее обоснованным техническим решением. Концепция RAG была формализована в работе Lewis et al. [1], где авторы показали, что модели, сочетающие параметрическую и непараметрическую память, генерируют более точные и фактически достоверные ответы по сравнению с системами, опирающимися исключительно на веса модели. Как отмечается в обзоре Gao et al. [3], ключевыми проблемами современных языковых моделей являются именно галлюцинации, устаревание знаний и непрозрачность рассуждений — RAG выступает признанным решением каждой из них. Прежде всего, система должна оперировать исключительно официальными документами НИУ ВШЭ, не допуская генерации ответов на основе параметров модели. Это обусловлено высокой ценой ошибки: сведения о сроках подачи документов, условиях поступления и требованиях к абитуриентам носят юридически значимый характер, и любое фактическое искажение — так называемая «галлюцинация» модели — может повлечь за собой серьёзные последствия для пользователя. RAG устраняет данный риск, привязывая каждый ответ к конкретному фрагменту источника. Помимо этого, нормативная документация приёмной кампании регулярно обновляется. В отличие от дообучения, архитектура RAG позволяет актуализировать информационную базу без какого-либо вмешательства в параметры модели — достаточно обновить документы в хранилище. Совокупность указанных факторов определила выбор RAG в качестве архитектурной основы разрабатываемой системы.

3 Архитектура системы

Реализованная система представляет собой расширенный RAG-пайплайн, сочетающий два дополняющих друг друга метода улучшения запроса: переформулирование (Query Rewriting) и декомпозицию (Query Decomposition). Необходимость такого подхода обусловлена семантическим разрывом между разговорным стилем пользовательских запросов и официально-деловым регистром нормативных документов. Как показано в работе [2], прямой поиск по исходному запросу систематически уступает подходу «переформулировать – извлечь – ответить» (Rewrite-Retrieve-Read), поскольку позволяет привести запрос к семантическому пространству базы знаний. Обработка запроса выполняется в два последовательных этапа. На первом этапе применяется детерминированное лексическое переформулирование на основе регулярных выражений для нормализации студенческого сленга. На втором этапе языковая модель YandexGPT одновременно переформулирует запрос и разбивает его на подзапросы. Данная стратегия декомпозиции опирается на метод Multi-Query Retrieval, который эффективно решает проблему чувствительности эмбедингов к точным формулировкам, генерируя различные семантические вариации одного вопроса. Для каждого подзапроса выполняется независимый поиск в векторном хранилище ChromaDB. Использование модели multilingual-E5 обосновано её высокой производительностью в задачах извлечения текстов на разных языках, подтвержденной в сравнительных бенчмарках [4]. Полученные чанки объединяются и дедуплицируются. Критически важным этапом является переранжирование (Reranking). Согласно экспериментальному исследованию ARAGOG [5], использование внешних ранкеров является наиболее эффективным методом повышения точности (Precision) RAG-систем. Применение Cross-Encoder архитектуры позволяет более детально оценить релевантность фрагмента контекста за счёт механизма полного внимания между токенами вопроса и документа [6], что нивелирует недостатки стандартного косинусного сходства векторов. Итоговый контекст передаётся в генеративную модель для формирования финального ответа.

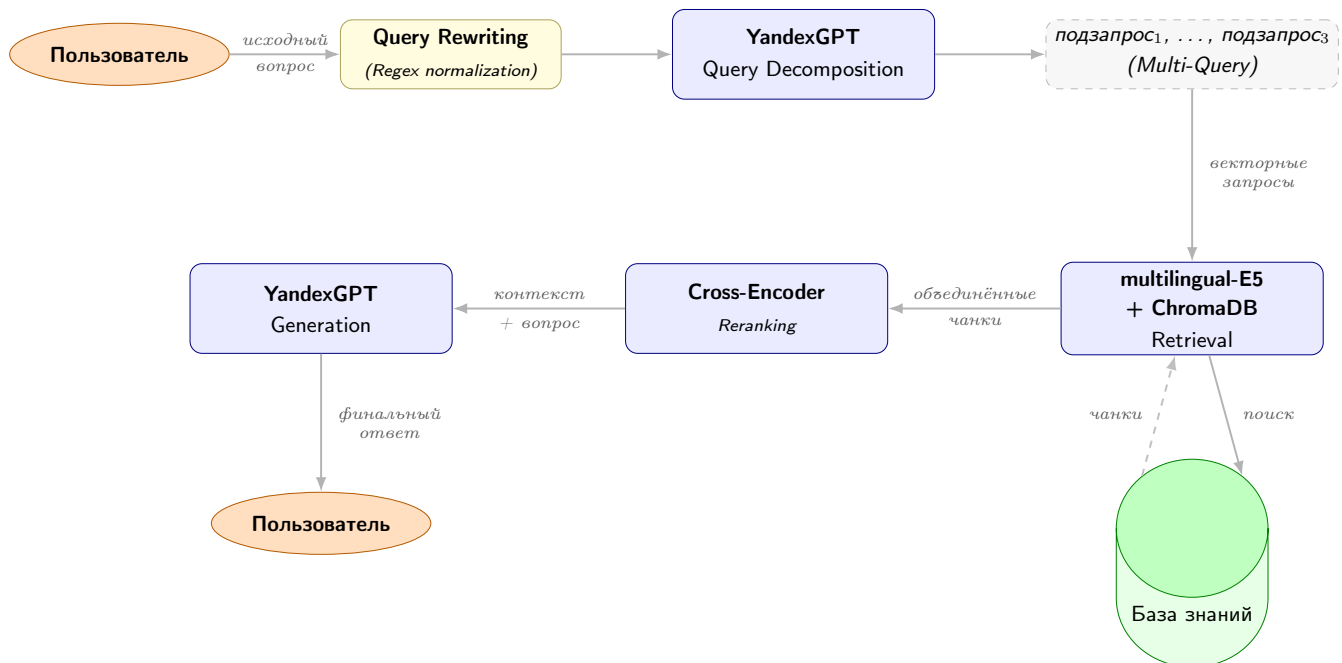


Рис. 1: Схема работы RAG-системы с переформулированием и декомпозицией запроса

4 Ход работы

В данном разделе описывается процесс практической реализации системы. Работа разделена на несколько ключевых этапов: подготовка корпуса знаний, выбор и настройка модели векторных представлений, построение индексного хранилища, реализация многостадийного retrieval-пайплайна и его интеграция с большой языковой моделью.

4.1 Описание датасетов

4.1.1 База знаний

База знаний сформирована из 13 тематических документов в формате Markdown общим объёмом около 138 000 символов. Каждый документ охватывает одну предметную область, что позволяет контролировать полноту покрытия тем. Структура базы знаний представлена в таблице.

Файл	Тема	Тип
fkn_programs_rag.md	Программы и минимальные баллы ЕГЭ	Нормативный
fkn_achievements_rag.md	Индивидуальные достижения	Нормативный
fkn_discounts_rag.md	Скидки на обучение	Нормативный
fkn_dormitory_rag.md	Общежитие	Нормативный
fkn_contract_rag.md	Договор об оказании услуг	Нормативный
fkn_military_rag.md	Военная кафедра	Нормативный
fkn_academic_process_rag.md	Академический процесс	Нормативный
fkn_foreign_rag.md	Иностранные граждане	Нормативный
fkn_ovz_rag.md	Абитуриенты с ОВЗ	Нормативный
fkn_timeline_rag.md	Сроки подачи документов	Нормативный
fkn_tuition_rag.md	Стоимость обучения	Нормативный
fkn_summary_rag.md	Сводная информация по программам	Нормативный
ЧАТ_ФКН_2025.md	FAQ Telegram-канала ФКН	FAQ

Таблица 1: Состав базы знаний

Документы собирались из двух источников: официальные PDF-документы НИУ ВШЭ (Правила приёма 2026 и приложения к ним с [официального сайта](#)) и Telegram-канал Студенческого совета ФКН за 2024–2026 годы. Текст извлекался с помощью библиотеки pdfplumber, после чего структурировался в формат Markdown с использованием языковой модели: нормативные документы приводились к иерархической разметке по разделам, а сообщения Telegram-канала – к FAQ-структуре с явным выделением вопроса и ответа. После автоматизированной обработки весь контент проходил этап ручной верификации для устранения возможных галлюцинаций модели и проверки фактической точности данных.

4.1.2 Тестовый датасет

Для оценки системы сформирован датасет из 300 пар «вопрос – эталонный ответ». Структура каждой записи описана в таблице.

Поле	Тип	Описание
id	str	Идентификатор записи (Q01–Q300)
question	str	Текст вопроса пользователя
answer	str	Эталонный ответ из официальных документов
category	str	Тематическая категория (см. табл. 3)
route	str	Ожидаемый маршрут: rag, clarification, blocked, faq
source_doc	str	Документ базы знаний, содержащий эталонный ответ

Таблица 2: Поля тестового датасета

Датасет разбит на несколько категорий, равномерно распределённых по трём частям, где каждая следующая часть расширяет тематическое покрытие. Состав категорий приведён в таблице.

Тип / тема вопроса	Маршрут	Кол-во
<i>Тематические вопросы</i>		
Программы, поступление, баллы ЕГЭ	rag	26
Олимпиады и индивидуальные достижения	rag	25
Сроки, документы, договор	rag	25
ОВЗ и иностранные граждане	rag	30
Стоимость обучения и скидки	rag	24
Общежитие	rag	14
Военная кафедра	rag	11
Академический процесс	rag	19
<i>Специальные типы</i>		
Сленговые и разговорные формулировки	rag	30
Сложные и составные вопросы	rag	41
<i>Фильтруемые запросы</i>		
Грубые / нецензурные	blocked	35
Офтопик-запросы	blocked	20
Итого		300

Таблица 3: Состав и структура тестового датасета

Вопросы составлялись вручную на основе анализа Telegram-чатов абитуриентов ФКН за 2024 и 2025 годы. Эталонные ответы находились в официальных документах и фиксировались вручную. Тестовая выборка не входила в базу знаний и использовалась исключительно для оценки. Датасет опубликован в открытом доступе в формате Markdown¹

4.2 Выбор метрик

Оценка системы проводилась независимо по двум компонентам: качеству поиска и качеству генерации ответа. Для оценки качества поиска использовались метрики Hit@K, MRR@5, Precision@K и Context Recall. **Hit@K** фиксирует долю запросов, для которых хотя бы один релевантный фрагмент попал в топ- K результатов. **MRR@5** (Mean Reciprocal Rank) учитывает позицию первого релевантного чанка, отражая стабильность ранжирования. **Precision@K** определяет долю полезной информации среди K возвращённых фрагментов. **Context Recall** оценивает, какая часть информации из эталонного ответа фактически присутствует в извлечённом контексте. Критерием релевантности чанка служила доля токенов эталонного ответа, присутствующих в тексте фрагмента: порог составлял 0,25². Для оценки качества генерации использовались метрики семейства ROUGE. **ROUGE-1** измеряет перекрытие униграмм между сгенерированным и эталонным ответом, **ROUGE-L** – длину наибольшей общей подпоследовательности, что позволяет учитывать порядок слов.

4.3 Подготовка данных и метод чанкирования

4.3.1 Предобработка текстов

Извлечённый из PDF-файлов «сырой» текст содержал колонтитулы, номера страниц и артефакты переноса строк. Очистка выполнялась в два шага. На первом шаге применялась библиотека *pdfplumber*, которая корректно обрабатывает многоколоночные таблицы и извлекает структурированный текст без артефактов разрыва строк. На втором этапе для приведения документов к единому формату разметки Markdown использовалась большая языковая модель. На следующем этапе нормативные документы были преобразованы в иерархическую структуру с использованием заголовков уровней #, ## и ###. Материалы Telegram-канала были трансформированы в FAQ-структуру, где вопросы выделялись полужирным начертанием, а ответы отделялись логическим разрывом строки. Важным этапом подготовки стала последующая ручная сверка полученных данных.

¹https://github.com/aveliaev/RAG_HSE

²Значение выбрано эмпирически: порог 0,1 оказался слишком низким, а порог 0,5 — слишком строгим, поэтому решил выбрать что-то между

4.3.2 Метод чанкирования

Структура документов в базе знаний разная: часть материалов представлена в виде FAQ с готовыми парами «вопрос – ответ», а часть – как нормативные документы с разделением разделов. Поэтому я отказался от разделения текста на фиксированные куски и применил структурное чанкирование, то есть разбиение с учетом разметки документа. Это было сделано для того чтобы каждый чанк содержал логически законченную мысль. Пайплайн сам выбирает один из трёх алгоритмов в зависимости от формата файла. Если в документе хотя бы пять строк с жирным заголовком, то каждая пара «вопрос – ответ» идёт в отдельный чанк. Если есть подзаголовки, границы режутся по этим тегам, заголовки верхних уровней добавляются в текст чанка для контекста. Во всех остальных случаях документ делится по заголовкам. При загрузке в ChromaDB к каждому чанку добавляется префикс `passage:`, а к поисковым запросам – `query:`, согласно рекомендации авторов модели multilingual-E5 [4].

4.4 Выбор модели векторного представления текста

Для формирования эмбедингов текстовых данных из библиотеки SentenceTransformers были рассмотрены три предобученные модели: *paraphrase-multilingual-MiniLM-L12-v2* (118 М параметров), *intfloat/multilingual-e5-base* (278 М параметров) и *BAAI/bge-m3* (570 М параметров). Сравнительный эксперимент проводился на выборке из 50 тематических вопросов о поступлении в режиме чистого векторного поиска – без реранжирования и нормализации запросов – чтобы изолировать вклад именно модели эмбедингов. В качестве метрик использовались стандартные показатели качества информационного поиска: Hit Rate@3 и Mean Reciprocal Rank (MRR@3). Hit Rate@3 – доля запросов, где нужный чанк попал в тройку лучших; MRR@3 – насколько высоко в среднем стоит правильный чанк в выдаче.

Модель	Hit Rate@3	MRR@3
paraphrase-multilingual-MiniLM-L12-v2	0,660	0,583
BAAI/bge-m3	0,740	0,671
intfloat/multilingual-e5-base	0,820	0,748

Таблица 4: Сравнение моделей векторного представления текста ($n = 50$)

По итогам эксперимента я выбрал *intfloat/multilingual-e5-base*, потому что она показала лучшие результаты по обоим метрикам. Стоит заметить, что *BAAI/bge-m3* при большем числе параметров всё равно проиграла, потому что она плохо работает с разными форматами запросов. У E5 эту проблему решают явные префиксы `query/passage`, которые хорошо работают на русскоязычных текстах [4].

4.5 Выбор модели переранжирования

Векторный поиск на основе эмбедингов позволяет быстро найти семантически близкие чанки, однако bi-encoder архитектура имеет принципиальное ограничение: запрос и документ кодируются независимо, без учёта их взаимного контекста. Для устранения этого недостатка в пайплайн добавлен этап переранжирования (reranking): на первом шаге векторный поиск извлекает расширенный пул кандидатов (в два раза больше целевого числа чанков), на втором шаге cross-encoder модель заново оценивает каждую пару (запрос, чанк) совместно и переставляет кандидатов в порядке убывания релевантности. Для реализации этапа переранжирования была выбрана модель *cross-encoder/mmarco-mMiniLMv2-L12-H384-v1* из библиотеки SentenceTransformers. Модель обучена на датасете mMARCO – многоязычной версии MS MARCO – и нативно поддерживает русский язык, что критично для данной задачи. Эксперимент проводился на той же выборке из 50 вопросов; сравнивалась работа системы без переранжирования и с ним при фиксированной модели эмбедингов *multilingual-e5-base*.

Конфигурация	Hit Rate@3	MRR@3
Без переранжирования	0,820	0,748
+ mmarco-mMiniLMv2-L12-H384-v1	0,860	0,801

Таблица 5: Влияние переранжирования на качество поиска ($n = 50$)

Добавление cross-encoder reranker’a дало прирост Hit Rate@3 и MRR@3. Рост MRR особенно значим: он свидетельствует о том, что релевантный чанк стабильно поднимается на первые позиции выдачи, напрямую влияя

на качество контекста, передаваемого в генеративную модель. Данный эффект согласуется с выводами обзора [3], согласно которым переранжирование является одним из наиболее эффективных методов повышения точности RAG-систем.

4.6 Сравнительный анализ конфигураций пайплайна

Для оценки вклада каждого компонента системы проведено накопительное аблационное исследование: конфигурации тестируются последовательно, каждая следующая добавляет один компонент к предыдущей. Оценка проводилась на 245 вопросах тестового датасета (за исключением фильтруемых запросов). Метрики поиска вычислялись по топ-5 результатам: Hit@1, Hit@3, Hit@5, MRR@5, Precision@3 и Context Recall; дополнительно фиксировалась задержка retrieval-этапа (среднее и P95). Результаты представлены в таблице

Таблица 6: Метрики качества при накопительном добавлении компонентов ($n = 245$)

Конфигурация	Hit@1	Hit@3	Hit@5	MRR@5	P@3	CR	Lat _{mean}	Lat _{P95}
Base RAG	0,714	0,849	0,906	0,789	0,446	0,667	16 мс	17 мс
+ Reranker	0,796	0,931	0,959	0,862	0,533	0,706	234 мс	408 мс
+ Regex Rewrite	0,820	0,922	0,955	0,872	0,550	0,701	248 мс	462 мс
+ LLM Decomp.	0,784	0,927	0,939	0,852	0,543	0,700	1250 мс	1707 мс
+ FAQ Cache	–	–	–	–	–	0,809	786 мс	1580 мс
+ Agentic Loop	0,820	0,943	0,967	0,882	0,546	0,713	3269 мс	6969 мс

Retrieval-метрики для конфигурации с FAQ-кешем не применимы: при попадании в кеш (35,5% запросов) система возвращает ответ напрямую, минуя retrieval-этап, поэтому в данном случае они не представляют интереса.

4.6.1 Конфигурация 1: Base RAG

В качестве базовой конфигурации используется стандартный RAG-пайплайн [1]: пользовательский запрос напрямую передаётся в векторное хранилище ChromaDB, оснащённое эмбедами модели intfloat/multilingual-e5 base [4], и возвращаются пять наиболее семантически близких фрагментов. Базовая конфигурация достигает Hit@5 = 0.906, что свидетельствует о том, что в 90.6% случаев релевантный фрагмент присутствует среди пяти возвращённых кандидатов. Вместе с тем Hit@1 = 0.714 указывает на существенный разрыв между первой позицией и топ-5: более чем в четверти запросов наиболее релевантный фрагмент не попадает на первое место, что снижает качество контекста, передаваемого генеративной модели. Задержка retrieval-этапа составляет 16 мс – наименьшая среди всех конфигураций, обусловленная компактным размером базы знаний (216 чанков).

4.6.2 Конфигурация 2: Добавление Reranker

Основным ограничением архитектуры, лежащей в основе векторного поиска, является раздельное кодирование запроса и документа без учёта их взаимного контекста. Для устранения этого недостатка к пайплайну добавлен этап переранжирования на основе cross-encoder модели cross-encoder/mmarco-mMiniLMv2-L12-H384-v1, обученной на многоязычной версии датасета MS MARCO. Согласно исследованию ARAGOG [5], применение внешних ранкеров является наиболее эффективным методом повышения точности RAG-систем. Cross-encoder оценивает каждую пару (запрос, фрагмент) совместно через механизм полного внимания [6], что обеспечивает более точную оценку релевантности по сравнению с косинусным сходством векторов. Добавление ранкера даёт наибольший прирост среди всех компонентов: Hit@1 вырос с 0.714 до 0.796, Hit@3 – с 0.849 до 0.931, MRR@5 – с 0.789 до 0.862, Precision@3 – с 0.446 до 0.533. В данной конфигурации задержка выросла с 16 до 234 мс. Но рост MRR очень важный показатель, потому что теперь нужный чанк теперь чаще оказывается первым в выдаче.

4.6.3 Конфигурация 3: Добавление Rewrite

Главная проблема при поиске по регламентам в том, что абитуриенты пишут разговорным языком, а документы составлены в официально-деловом стиле. Чтобы сократить этот разрыв, я реализовал нормализацию на основе регулярных выражений, например, «*пми*» превращается в «*прикладная математика и информатика*». В отличие от LLM-переформулирования, такой подход не даёт галлюцинаций и не добавляет задержку.

Благодаря такому дополнению Hit@1 вырос до 0,820, MRR@5 – до 0,872, Precision@3 – до 0,550, задержка почти не изменилась (248 мс).

4.6.4 Конфигурация 4: Добавление LLM Decomposition

Одна из главных проблем эмбедингов – чувствительность к точным формулировкам. Для решения этой проблемы я реализовал Multi-Query: для каждого запроса генерируется несколько вариантов, чтобы найти больше релевантных чанков. Однако если посмотреть на метрики, они упали: Hit@1 снизился с 0,820 до 0,784, Hit@5 – с 0,955 до 0,939, MRR@5 – с 0,872 до 0,852, а задержка выросла в 5 раз (248 мс $\Rightarrow$ 1250 мс). Исключение – Hit@3, незначительно выросший с 0,922 до 0,927. Скорее всего это произошло потому, что вопросы и так близки к документам, и декомпозиция только добавляет шум. Тем не менее я решил оставить Multi-Query, потому что без него модель упускает детали на составных вопросах.

4.6.5 Конфигурация 5: Добавление FAQ-кеша

Для снижения нагрузки на языковую модель реализован трёхуровневый FAQ-кеш: точное совпадение по нормализованному хешу вопроса, нечёткое сопоставление по коэффициенту Жаккара и динамический кеш, пополняемый ответами, подтверждёнными пользователями. При попадании в кеш (35,5% запросов) система возвращает ответ напрямую, минуя retrieval-этап, поэтому retrieval-метрики для данной конфигурации не применимы – в таблице они обозначены прочерками. Среднее время ответа сократилось с 1250 мс до 786 мс, при этом 35,5% запросов обрабатываются мгновенно. Высокий Context Recall = 0,809 объясняется тем, что кешированные ответы являются точными эталонными парами датасета.

4.6.6 Конфигурация 6: Добавление Agentic Loop

Агентный цикл устроен так: после первого ответа модель проверяет, хватило ли ей контекста, и если нет – делает ещё один поиск с уточнённым запросом. Это продолжается до тех пор, пока модель не решит, что контекста достаточно, или не кончатся итерации. Агентный подход помогает в составных вопросах, где нужны чанки из разных разделов. Если смотреть на метрики, то можно увидеть, что Hit@3 вырос с 0,927 до 0,943, Hit@5 – с 0,939 до 0,967, MRR@5 – с 0,852 до 0,882, а значение Hit@1 осталось на уровне 0,820. Заметим, что время retrieval этапа увеличилось с 786 мс до 3269 мс, это связано с тем, что он еще и проверяет ответ.

4.6.7 Итоговый пайплайн

По результатам сравнительного анализа базовая итоговая конфигурация системы включает пять компонентов: Base RAG + Reranker + Rewrite + LLM Decomposition + FAQ-кеш. Реранкер обеспечивает наибольший прирост retrieval-метрик. LLM-декомпозиция сохранена в итоговой конфигурации, поскольку повышает полноту ответов на составные вопросы – эффект, который retrieval-метрики не фиксируют. Включение агентного цикла является опциональным и определяется требованиями конкретного развёртывания. Его можно включить или отключить с помощью флага в конфигурационном файле системы. Агентный цикл рекомендуется включить, когда задержка не критична, а в потоке запросов преобладают составные многоступенчатые вопросы: он улучшает Hit@3, Hit@5 и MRR@5. В сценариях с жёсткими требованиями к задержке (интерактивный чат, высокая нагрузка) агентный цикл следует отключить – пятикомпонентная конфигурация обеспечивает приемлемое качество при значительно меньших задержках.

4.7 Архитектура Telegram-бота

Telegram-бот реализован на библиотеке python-telegram-bot v21 и служит пользовательским интерфейсом над RAG-системой. Входящий запрос последовательно проходит через несколько фильтров и направляется в один из обработчиков. Такая конвейерная архитектура позволяет отвечать на типовые вопросы мгновенно, не обращаясь к языковой модели, и тем самым снижает задержку и стоимость обслуживания запросов. Обработка каждого сообщения начинается с проверки ограничителя частоты запросов: если пользователь превысил лимит в 12 обращений за 60 секунд, запрос отклоняется без дальнейшей обработки. Следующим шагом срабатывает фильтр контента: сообщения с нецензурной лексикой получают вежливый отказ, а офтопик-запросы – просьбы написать код, сочинение и т. п. – отклоняются с пояснением о специализации бота. Затем запрос проверяется по трёхуровневому FAQ-кешу: первый уровень выполняет точное совпадение по нормализованному хешу вопроса, второй – нечёткое сопоставление по коэффициенту Жаккара, третий – поиск в динамическом кеше, который автоматически пополняется RAG-ответами, подтверждёнными пользователями

через оценку «лайк». Если кеш не дал результата, система проверяет, достаточно ли контекста для поиска: например, при вопросе о минимальных баллах без указания программы бот запрашивает уточнение. Только при отсутствии результата на всех предыдущих уровнях запрос передаётся в Agentic RAG – декомпозиция, поиск в ChromaDB, переранжирование и генерация ответа. Каждый RAG-ответ сопровождается кнопками оценки «лайк / дизлайк»; положительная оценка автоматически добавляет пару вопрос–ответ в динамический кеш для ускорения последующих обращений. Если же пользователь поставил дизлайк ответу, который был получен из динамического кеша, эта запись немедленно удаляется из кеша, вносится в чёрный список и помещается во временный словарь для последующей ручной проверки – при повторном вопросе система обратится напрямую к RAG-пайплайну.

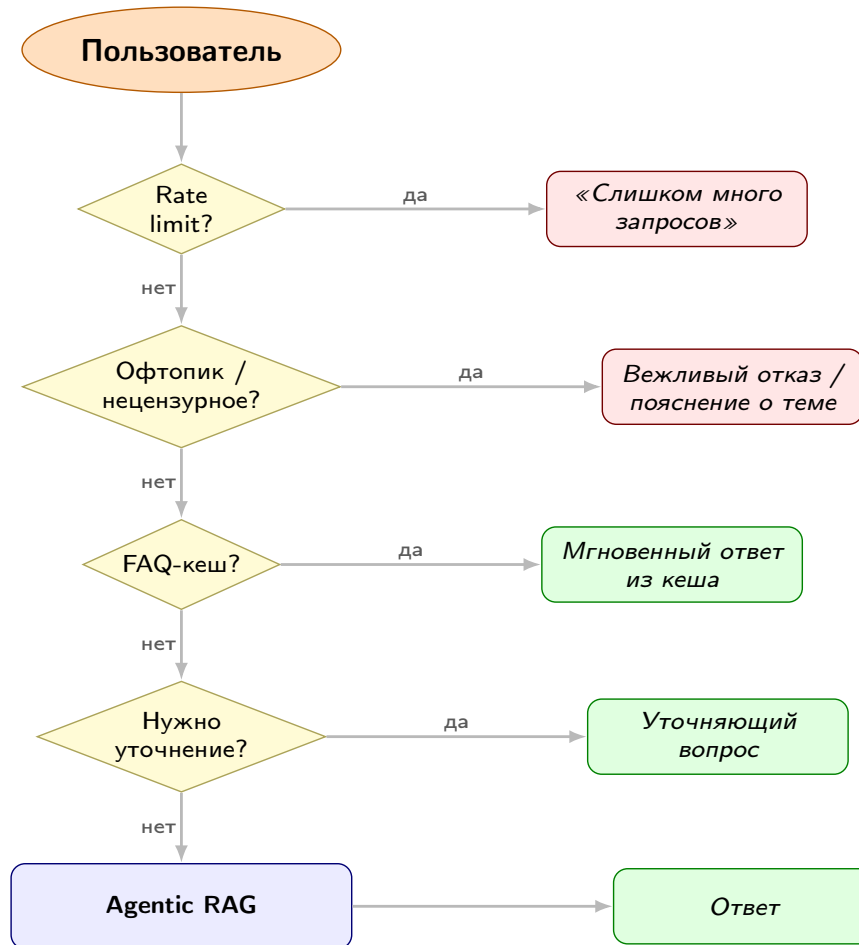


Рис. 2: Схема маршрутизации запросов в Telegram-боте

4.8 Результаты и анализ

В рамках данной работы разработан Telegram-бот для автоматического консультирования абитуриентов факультета компьютерных наук НИУ ВШЭ. Бот реализован на основе RAG-архитектуры и предоставляет ответы на вопросы о правилах поступления, проходных баллах, льготных квотах, общежитии и учебном процессе, опираясь на актуальную базу знаний из официальных документов ФКН. Система развёрнута в мессенджере Telegram и доступна в режиме реального времени; в качестве генеративной модели используется YandexGPT с резервным переключением на Gtoq при недоступности основного провайдера. Экспериментальная оценка проводилась на контрольной выборке из 300 запросов, собранных независимо от обучающего датасета. Из них 237 запросов (79%) были классифицированы как целевые и обработаны RAG-пайплайном; 55 запросов (18,6%) отсечены фильтром безопасности, 7 запросов (2,4%) потребовали уточнения контекста. Фильтр контента показал точность 100% на тестовом наборе провокационных запросов. Итоговые показатели представлены в таблице.

Группа	Метрика	Значение
<i>Retrieval</i>	Hit@1	0,768
	Hit@3	0,916
	Hit@5	0,949
	MRR@5	0,844
	Precision@3	0,547
	Context Recall	0,699
<i>Generation</i>	ROUGE-1	0,453
	ROUGE-L	0,403
<i>Performance</i>	Latency (mean)	17,2 с
	Latency (p95)	18,0 с

Таблица 7: Итоговые метрики качества системы ($n = 237$)

Значение $\text{Hit@5} = 0,949$ означает, что в 95% случаев релевантный фрагмент присутствует среди кандидатов, передаваемых генеративной модели; $\text{MRR@5} = 0,844$ говорит о том, что нужный фрагмент стабильно оказывается высоко в выдаче. Стоит отметить, что значения $\text{ROUGE-1} = 0,453$ и $\text{ROUGE-L} = 0,403$ получились невысокими не потому что ответы плохие, а потому что модель отвечает развёрнуто, а эталоны короткие — метрика лексического перекрытия это не учитывает. Среднее время ответа 17,2 с обусловлено вызовом генеративной модели; при включённом Agentic RAG оно будет выше. Retrieval-этап с учётом FAQ-кеша занимает менее 1 с.

5 Заключение

В рамках выполненного программного проекта был спроектирован и реализован Telegram-бот для ответов на вопросы абитуриентов факультета компьютерных наук НИУ ВШЭ. В ходе работы были решены все поставленные задачи и получены следующие практические выводы. Эксперименты с шестью конфигурациями пайплайна показали, что базовая схема RAG не обеспечивает достаточной точности для задач приёмной кампании ввиду специфики нормативных документов и особенностей пользовательского ввода. Наибольший прирост поисковых метрик обеспечил Cross-Encoder: показатель Hit@1 увеличился с 0,714 до 0,796, а MRR@5 — с 0,789 до 0,862. Нормализация с помощью регулярных выражений позволила сократить разницу между официальными документами и разговорным стилем абитуриентов. Разработанный трёхуровневый FAQ-кеш обеспечил мгновенную обработку 35,5% запросов, что существенно снижает стоимость проекта. Итоговые испытания на контрольном датасете из 300 пар вопросов и ответов зафиксировали значение метрики Hit@5 на уровне 0,949: в 95% сценариев система находит корректные данные в топ-выдаче. Фильтр нецелевого контента показал точность 100% на тестовом наборе провокационных запросов. Разработанный протокол оценки и структурированная база знаний создают надёжный фундамент для дальнейшего масштабирования проекта и внедрения более сложных сценариев взаимодействия.

6 Использование ИИ

В процессе работы над проектом использовались инструменты на основе больших языковых моделей. Для отладки кода и диагностики ошибок применялись [Claude](#) и [Gemini](#): при возникновении сбоев модели помогали установить их причину — в частности, разграничить собственные ошибки в логике кода и проблемы окружения, вызванные обновлением зависимостей или изменением конфигурации. С помощью тех же инструментов были разработаны вспомогательные дашборды для удобного отслеживания хода экспериментов. На этапе формирования базы знаний языковые модели использовались для парсинга и структурирования открытых документов в формат Markdown; все результаты автоматизированной обработки проходили обязательную ручную верификацию на фактическую точность. При написании текста курсовой работы модели применялись для проверки грамматики и улучшения форматирования. Для изучения научных статей использовался сервис [NotebookLM](#), который позволил эффективно работать с источниками и выделять ключевые идеи. Постановка задач, проектирование экспериментов, интерпретация результатов и итоговые выводы выполнены автором самостоятельно под руководством научного руководителя.

Список литературы и источников

- [1] Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [Электронный ресурс] // Advances in Neural Information Processing Systems. — 2020. — Т. 33. — С. 9459–9474. — URL: <https://arxiv.org/abs/2005.11401> (дата обращения: 10.01.2026).
- [2] Ma X. et al. Query Rewriting for Retrieval-Augmented Large Language Models [Электронный ресурс] // arXiv.org. — 2023. — URL: <https://arxiv.org/abs/2305.14283> (дата обращения: 11.05.2026).
- [3] Gao Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey [Электронный ресурс] // arXiv.org. — 2023. — URL: <https://arxiv.org/abs/2312.10997> (дата обращения: 09.05.2026).
- [4] Wang L. et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training [Электронный ресурс] // arXiv.org. — 2022. — URL: <https://arxiv.org/abs/2212.03533> (дата обращения: 09.05.2026).
- [5] Hansen L. et al. ARAGOG: Advanced RAG Output Grading [Электронный ресурс] // arXiv.org. — 2024. — URL: <https://arxiv.org/abs/2404.01037> (дата обращения: 09.05.2026).
- [6] Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks [Электронный ресурс] // arXiv.org. — 2019. — URL: <https://arxiv.org/abs/1908.10084> (дата обращения: 09.05.2026).